

Academic Evidence Investigator

An LLM-powered multi-agent planning agent for academic evidence

Unit 11 individual implementation of the Unit 6 group design

Ruwan Perera

One goal in, a referenced briefing out



Goal

Research question from a postgraduate researcher

Plan

Sub-questions and search queries

Retrieve

Semantic Scholar and Crossref

Appraise

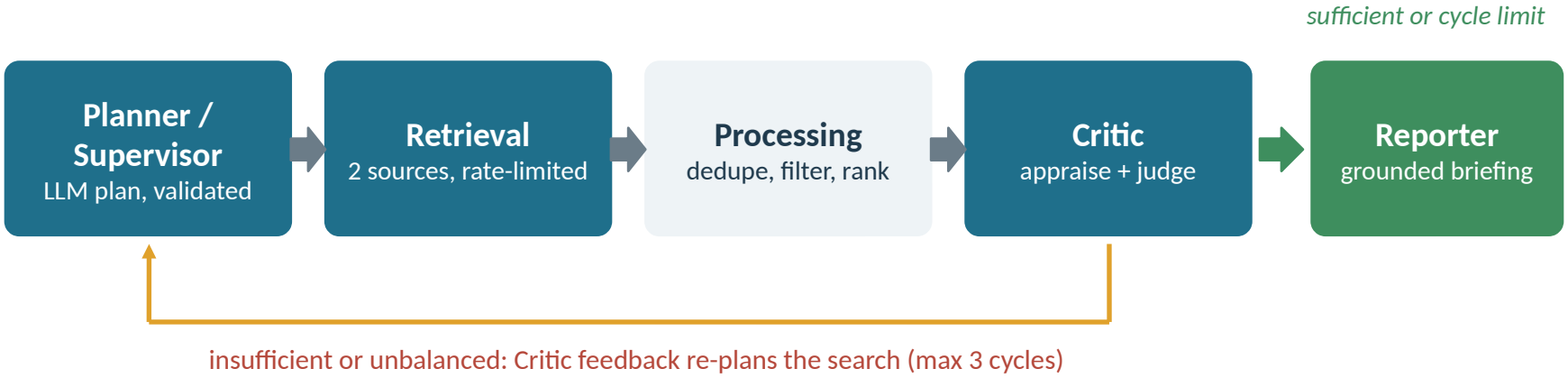
Finding, design, stance, limitations

Report

Grounded briefing with Harvard references

Autonomous re-planning: if the Critic judges the evidence thin or one-sided, the agent decides by itself to search again with a revised plan, up to a fixed cycle limit.

LangGraph realises the proposed architecture



Shared ResearchState
Beliefs: papers, appraisals, verdict. Intention: the Planner's current queries (BDI; Archibald et al., 2024). Append-only audit trail.

Why LangGraph
Explicit conditional edge and bounded loop: the evaluator-optimiser pattern (LangChain, no date). ReAct-style alternation of reasoning and acting (Yao et al., 2023).

The LLM proposes, Python verifies

Agent	LLM does (Qwen2.5-7B-Instruct)	Python guarantees
Planner	Decomposes the question, writes queries	Schema check, no repeated queries, keyword fallback
Retrieval	Nothing	Rate limits, retries only transient errors, logs outages
Processing	Nothing, by design	DOI then title + author dedupe, relevance ranking
Critic	Appraises abstracts in one batched call	Drops invented paper ids; stop rule is explicit thresholds
Reporter	Drafts the briefing	Every finding must cite a retrieved paper

Rule-based stopping

Thresholds, not the model, decide whether to search again (Ji et al., 2023)

Grounded reporting

Citations to papers never retrieved are stripped (Lewis et al., 2020; Huang et al., 2025)

Graceful degradation

Every LLM step has a fallback; the audit trail records which ran

Explicit offline mode

Reproducible demos and tests, never a silent fallback

Code and executables

```
evidence_investigator/
  graph.py       LangGraph wiring, re-plan loop
  state.py       shared ResearchState
  agents/        the four agents
  sources/       Semantic Scholar, Crossref
  llm.py         Qwen client, JSON validation
  processing.py  dedupe, filter, rank
  storage.py     SQLite run history
  web/           Flask interface
tests/           83 tests at four levels
evidence/        logs, screenshots, briefings
```

Command line

```
python -m evidence_investigator "<question>" -v
```

Briefing plus the audit trail of every agent step

Web interface

```
python -m evidence_investigator.web.app
```

Form, briefing, evidence table and run history

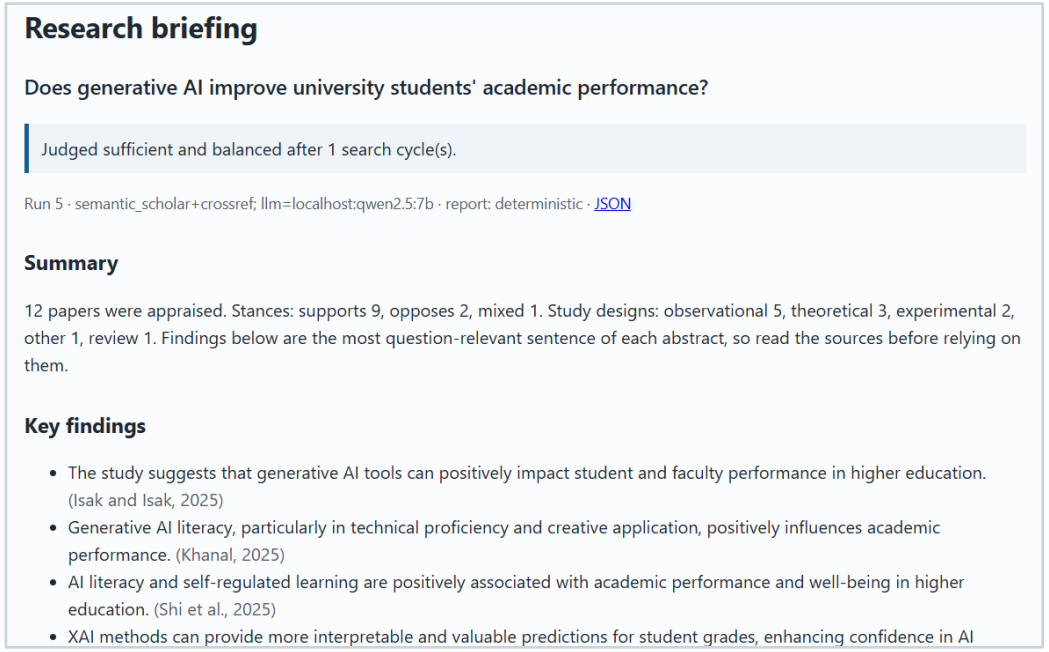
Tests

```
python -m pytest --cov=evidence_investigator
```

83 passed, 0 failed, 96% coverage

Live mode uses Qwen2.5-7B under Ollama (LLM_BASE_URL); --offline runs the same code on a bundled corpus. Setup in README.md, sections 3 and 4.

Demonstration: from question to briefing



Research briefing

Does generative AI improve university students' academic performance?

> Judged sufficient and balanced after 1 search cycle(s).

Run 5 · semantic_scholar+crossref; llm=localhost:qwen2.5:7b · report: deterministic · JSON

Summary

12 papers were appraised. Stances: supports 9, opposes 2, mixed 1. Study designs: observational 5, theoretical 3, experimental 2, other 1, review 1. Findings below are the most question-relevant sentence of each abstract, so read the sources before relying on them.

Key findings

- The study suggests that generative AI tools can positively impact student and faculty performance in higher education. (Isak and Isak, 2025)
- Generative AI literacy, particularly in technical proficiency and creative application, positively influences academic performance. (Khanal, 2025)
- AI literacy and self-regulated learning are positively associated with academic performance and well-being in higher education. (Shi et al., 2025)
- XAI methods can provide more interpretable and valuable predictions for student grades, enhancing confidence in AI

Evidence status
Written by Python from the Critic's verdict, never by the LLM

Key findings
Each cites retrieved papers in Harvard in-text style

Evidence table
Design, stance and relevance per paper

Audit trail + SQLite
Every search, re-plan and decision is stored and viewable

Demonstration: the Critic drives re-planning

```python
def route_after_critic(state: ResearchState) -> str:
    # The stop decision is a rule, not the model's opinion of its own work:
    # self-reflection reduces but does not remove hallucination (Ji et al., 2023).
    verdict = state["verdict"]
    if verdict.sufficient or state["cycle"] >= max_cycles:
        return "report"
    return "replan"

graph.add_conditional_edges("critic", make_router(max_cycles),
                            {"replan": "planner", "report": "reporter"})
```

evidence_investigator/graph.py: the conditional edge is the agent's decision point

```
cycle 1  critic    insufficient: 2 relevant papers (need 5); designs not varied
cycle 2  planner   re-plan: '... randomised controlled trial', '... meta-analysis'
cycle 2  critic    insufficient: still 2 relevant papers
cycle 3  planner   re-plan: broadened to 'retrieval reduce'
cycle 3  reporter  cycle limit reached: briefing marked provisional
```

What it did: offline run of "Does retrieval reduce hallucination?" (evidence/run-offline-max-cycles.log). By contrast, the live generative AI question was judged sufficient after one cycle.

Testing: 83 tests, no network needed

83

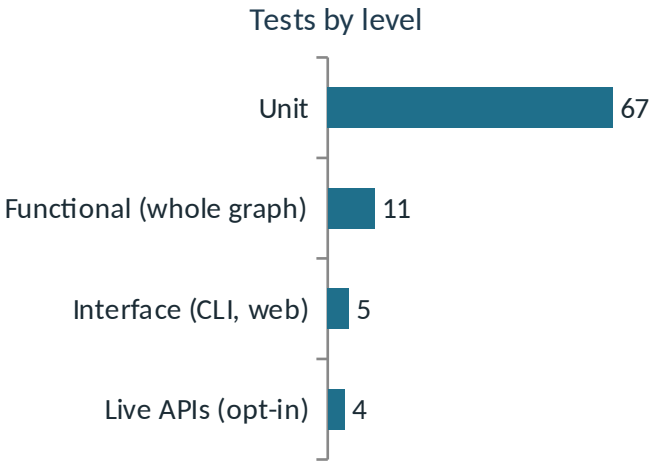
tests passed

0

failures

96%

line coverage



4 live tests skip unless RUN_LIVE_TESTS=1

Sources and the LLM are injected, so tests use scripted fakes and synthetic API fixtures.

Functional tests prove

- Critic triggers a re-plan
- Loop stops at the cycle limit
- One source down, run continues
- Cross-source duplicates merged
- Invented citation removed
- Total LLM outage still reports
- Hostile HTML is escaped

How it developed: build, test, refine, run live

1 Design

Unit 6 group proposal

- LangGraph over CrewAI
- Qwen2.5-7B-Instruct
- Two literature sources
- Critic-driven re-planning

Design agreed

2 Build offline-first

Incremental commits

- Schemas and shared state
- Sources and LLM client
- Agents in a LangGraph graph
- Flask interface
- Tests and evidence

Runs end to end

3 Test and refine

R1 to R5

- R1 synonyms: 2 to 10 papers
- R2 neutral is not bias
- R3 SQLite leaks closed
- R4 no retry on 404
- R5 Harvard order

76 tests, 97% coverage

4 Run live on Windows

R6 to R9, via pull requests

- R6 Qwen local via Ollama
- R7 longer 429 back-off
- R8 per-item validation
- R9 JSON mode

83 tests; live tests 3 of 4

Every step is a commit or pull request on GitHub, and each fix has a regression test (docs/TESTING.md).

Responsible use, limits and next steps

Legal, ethical, professional

- Provider rate limits and polite-pool etiquette respected
- Tokens only from the environment, never stored
- Gaps framed within retrieved papers only
- States it supports, not replaces, the researcher

Limits and next steps

- Heuristic appraisal is approximate without the LLM
- Abstracts only; term-overlap relevance
- Synchronous web requests, single user
- Next: embeddings, PostgreSQL, labelled evaluation

Key references: Archibald, B. et al. (2024) Software and Systems Modeling, 23. Huang, L. et al. (2025) ACM TOIS, 43(2). Ji, Z. et al. (2023) Findings of EMNLP. LangChain (no date) Workflows and agents. Lewis, P. et al. (2020) NeurIPS, 33. Yao, S. et al. (2023) ICLR. Full Harvard list in README.md.